

# LLM-as-a-Judge Reliability

## Phase 3 Combined Presentation

---

Parsa Gholami, Aria Alyasin, Mohsen Shirazi

Sharif University of Technology

## **Category 1: Benchmarks & Frameworks for Judges**

---

**Paper 3:**  
**M-Prometheus: A Suite of Open Multilingual  
LLM Judges**

# Introduction & Category Alignment

- **Category Context:** Research dedicated to creating standardized testing grounds to evaluate the evaluators themselves.
- **Core Objective:** Quantifying accuracy, consistency, and human-alignment of LLM judges across complex, non-English evaluation tasks.
- **Target Study:** Dissecting M-PROMETHEUS, a suite of open-weight multilingual LLM judges ranging from 3B to 14B parameters.
- **Practical Impact:** Bridging the severe automatic evaluation quality gap between English and non-English target languages.

# The Multilingual Evaluation Gap

- **The English-Centric Bias:** Most high-performance LLM-as-a-judge frameworks are optimized exclusively for English text processing.
- **Systematic Disparity:** Lack of reliable non-English evaluators creates a bottleneck, directly hindering the development of robust multilingual models.
- **Baseline Flaws:** Prior open-source multi-lingual attempts fail to support complex pairwise comparisons or inherit brittle capabilities solely from base pretraining.
- **Automatic Metric Failure:** Traditional scalar metrics like BLEU or COMET fail to provide explanatory, text-based reasoning for qualitative decisions.

# M-PROMETHEUS Architecture & Training Recipe

- **Model Foundation:** Built by fine-tuning state-of-the-art Qwen2.5-Instruct models across 3B, 7B, and 14B parameter scales.
- **Synthetic Paradigm:** Completely bypasses translated text training data, instead leveraging native multilingual feedback synthesized from scratch.
- **Data Infusion Strategy:** Combines massive custom-generated direct assessment and pairwise preference collections with cross-lingual machine translation eval data.
- **Dual Capability Mode:** Explicitly trained to operate flexibly across both reference-based and reference-free evaluation scenarios.

# Methodology Pipeline & Comparison

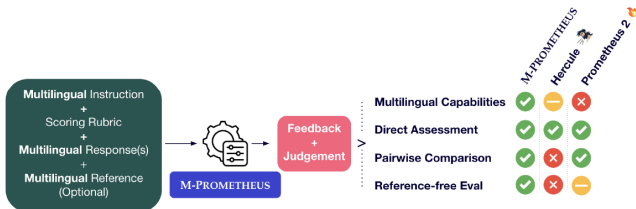


Figure 1: M-PROMETHEUS is a suite of open-weight multilingual LLM judges capable of providing reference-based and reference-free direct assessment and pairwise feedback.

# State-of-the-Art Evaluation Results

- **General Benchmark Dominance:** Outperforms all existing open-weight judges on comprehensive multilingual reward benchmarks covering over 20 languages.
- **Surpassing Proprietary Frontiers:** The 14B parameter model successfully outperforms massive closed-source baselines like GPT-4o on MM-Eval.
- **Literary Translation Excellence:** Demonstrates extreme precision on book excerpt translations, a cross-lingual domain where specialized metrics fail.
- **Generative Quality Amplification:** Directly enhances inference-time output accuracy through quality-aware decoding and selection mechanisms.



# Core Framework Strengths

- **SOTA Multilingual Benchmarking:** Delivers unmatched evaluation performance against both open-weight and proprietary models in massive non-English ecosystems.
- **Cross-Task Knowledge Transfer:** Demonstrates that integrating machine translation data significantly boosts a judge's general ability to detect language hallucinations.
- **Compute Efficiency at Scale:** Smaller variants like the 3B model deliver massive performance gains relative to their size, making localized deployment practical.
- **Native Capability Preservation:** Drastically enhances multilingual capabilities without degrading core performance or reliability on English benchmarks.

# Operational Limitations

- **Diminishing Returns on Scale:** Increasing parameter size to 14B yields structural improvements in theory, but lags behind smaller scales during active decoding tasks.
- **Language Saturation Constraints:** Directly optimized on only 6 language streams, relying heavily on the base model's zero-shot generalization for other target domains.
- **Explanatory Feedback Asymmetry:** While highly reliable at processing non-English prompts and inputs, the default long-form textual rationale remains written in English.

- **Data Engineering Shift:** Establishes native synthetic multi-lingual generation as a fundamentally superior alternative to translating preexisting English datasets.
- **Active Model Development:** Transforms the judge from a passive post-hoc benchmarking script into an active, inference-time text-improvement engine.
- **Future Framework Evolution:** Future reliability paradigms must integrate localized non-English explanation chains and native multi-lingual reasoning paths.

## **Category 2: Bias Quantification & Fairness Analysis**

---

## **Paper 5:**

**Any Large Language Model Can Be a Reliable  
Judge:Debiasing with a Reasoning-based Bias  
Detector**

# The Reliability Bottleneck: Systemic Pitfalls

- **Category Context:** Isolating and quantifying systemic prejudices (position, verbosity, bandwagon, and sentiment biases) that undermine automated trust.
- **Four Structural Vulnerabilities Under Analysis:**
  - **Verbosity Bias:** Prioritizing longer, low-quality answers over concise, mathematically accurate ones.
  - **Position Bias:** Inherent preference for choices based on their order slot.
  - **Bandwagon Bias:** Vulnerability to fabricated majority consensus indicators.
  - **Sentiment Bias:** Distorting scores based on enthusiastic stylistic tone.
- **Flaws of Existing Methods:** In-context prompting cannot break deep-seated cognitive patterns, while direct fine-tuning is impossible for closed endpoints.

# Dataset Construction & Distilled SFT Pipeline

- **Source Corpus Foundation:** Built upon curated samples from GSM8K (math), Arena (chat instruction pairs), and ScienceQA (multimodal science).
- **Data Curation Pipeline:**
  - Pairwise sets are modified to introduce targeted skews (e.g., stripping reasoning steps for verbosity, swapping choice order for position).
  - Ground-truth bias labels are mathematically computed by comparing biased evaluations against control judgments.
- **Reasoning Distillation Training Loop:**
  - A high-capacity Teacher LRM generates 1.67k expert reasoning traces explaining the flaws behind the judge's mistakes.
  - Hard validation filtering retains only traces matching the ground-truth label.
  - Base reasoning models (1.5B to 14B scales) undergo Supervised Fine-Tuning (SFT) jointly on all four bias types to learn core logical reasoning.

# RBD Framework Pipeline Overview

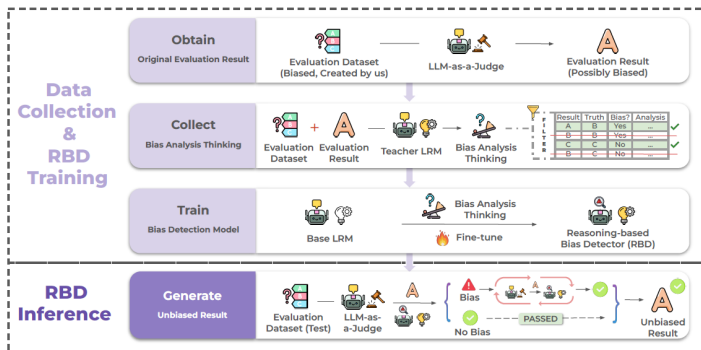


Figure 1: Overview of the Reasoning-based Bias Detector (RBD) framework. During **RBD inference**, it examines biased evaluation results produced by an LLM-as-a-Judge. If bias is identified, RBD generates a reasoning-based bias analysis to guide the LLM in reflecting on and potentially revising its evaluation; otherwise, the original judgment remains unchanged. To train RBD, we design a **data collection and distilled reasoning-based training pipeline**. We first construct a biased dataset containing specific types of bias and collect possibly biased evaluation results from the LLM evaluator. Then, a teacher Language Reasoning Model (LRM) produces bias analysis thinking based on the evaluation context. These analyses are filtered and used to fine-tune a base LRM into the final RBD model capable of identifying and correcting evaluation bias.

**Figure 1:** The end-to-end framework layout demonstrating dataset generation, validation filtering, and iterative feedback driven self-correction loops.



# Quantitative Evaluation & Key Results

- **Empirical Baseline Improvement:** Evaluated across 8 premier judges (GPT-4o, Claude-3.5, DeepSeek-V3, and LLaMA-3.1 series).
- **Core Performance Metrics:**
  - Integrating RBD-8B boosts evaluation accuracy by an average of **18.5%** and consistency by **10.9%**.
  - Outperforms prompt-based baselines by **12.8%** and out-of-the-box fine-tuned judge models by **17.2%**.
  - Smaller engines see massive jumps; LLaMA-3.1-8B accuracy surges up to **71.8%** on verbosity mitigation.
- **Stress Testing & Scalability:**
  - Scaling the RBD from 1.5B to 14B steadily enhances average accuracy gains (+6.5% to +15.4%).
  - Successfully mitigates multi-bias scenarios, securing recovery gains of +45% and +63.8% under overlapping vulnerabilities.

# Core Framework Strengths

- **True Cognitive Understanding:** Avoids exploiting synthetic data shortcuts by enforcing full paragraph-level comparative analysis before outputting labels.
- **Universal Agnostic Integration:** Operates entirely via text, facilitating seamless decoupling and deployment alongside proprietary, black-box commercial APIs.
- **Cross-Domain Generalization:** Maintains peak performance on unseen benchmarks (LLMBar, JudgeBench) and new domains (FactQA) without task-specific training.
- **Negligible Resource Overhead:** Leveraging optimized inference backends achieves an ultra-low latency of 1.5 seconds per sample.

# Operational Limitations

- **Cumulative Compute Cost:** While single-sample costs are fractional, multi-round iteration introduces cost aggregation when scaled across millions of requests.
- **Structural Constraint Scope:** Heavily tuned for multiple-choice and pairwise layouts; it does not natively adapt to purely open-ended generation score sheets.
- **Social Blind Spots:** Focuses strictly on surface-level format skews; it cannot explicitly target embedded demographic, cultural, or gender-based social biases.

- **The Scaled Stubbornness Trap:** Larger frontier models possess high internal certainty, making them structurally more stubborn and resistant to external correction.
- **The Authoritative Distillation Flaw:** Distilled, lightweight engines inherit the highly assertive tone of parents, masking an inability to deeply analyze foundational logic.
- **Future Paradigm Re-alignment:** Automated reliability cannot rely on internal alignment; trustworthy grading demands a hard transition toward decoupled, multi-agent referee structures.

## **Category 3: Scalability Limits & Benchmark Robustness**

---

**Paper 4:**  
**Tuning LLM Judge Design Decisions for**  
**1/1000 of the Cost**

- **Paper:** *Tuning LLM Judge Design Decisions for 1/1000 of the Cost*
- **Goal:** Replace expensive human evaluation with open-weight LLM judges — no quality loss, 1/1000 cost.
- **Category:** Scalability Limits & Benchmark Robustness.

# Problem Statement & Current Challenges

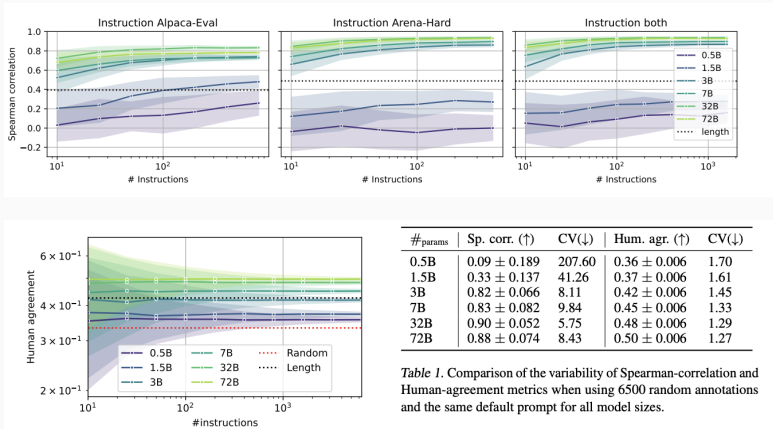
- **GPT-4 dependency:** sanctions risk, sudden behavior shifts, high cost.
- **Confounding factors:** model, prompt, hyperparameters change together (Alpaca-Eval, Arena-Hard).
- **Cognitive biases:** *length, position, self-enhancement.*



# Scaling Laws & Optimal Metric

- **Scaling up alone is not enough:** sub-7B judges act blindly, performing *worse* than the length baseline.
- **Spearman Correlation** (whole-leaderboard vs. human):
  - Noisy and costly; high coefficient of variation (CV) for small models.
  - Grows only with many more instructions — expensive to measure.
- **Human Agreement** (chosen metric):
  - High statistical stability, low noise ( $CV \approx 1.3$ ).
  - Stays *stationary* across instruction counts.
  - Distinguishes large models (32B, 72B) with far fewer, cheaper instructions.
- **Takeaway:** Human Agreement exposes Alpaca-Eval's simplicity vs. Arena-Hard's robustness, at a fraction of the cost.

# Scaling Laws — Figure 1 & Figure 2



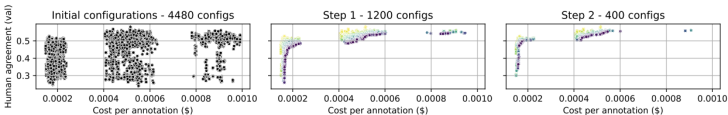
**Figure 2:** Performance scales dynamically with both model size and instruction count,

# Hyperparameter Search Space

- **4,480 configurations.**
- **Models (7 open-weight):** Llama 3.1 (8B, 70B), Qwen 2.5 (7B, 27B, 70B), Gemma 2 (9B, 27B).
- **Temperature:** [0.0, 0.01, 0.1, 1.0] + order-swap averaging.
- **Prompt formats (80):** single score, Likert, pair, multi-criteria; JSON vs. raw text.

# Smart Search Methodology

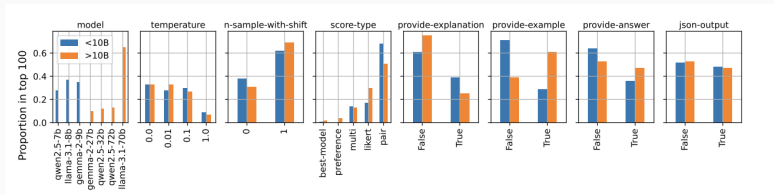
- **Multi-Fidelity (Successive Halving):**
  - 4,480 → 400 instructions.
  - 1,200 → 1,200 instructions.
  - 400 → 3,548 LMSys instructions.
- **Cost cut to  $\frac{1}{3}$**  of the traditional approach.
- **Multi-Objective:** non-dominated sort → Pareto front (accuracy vs. token cost).



**Figure 3:** Multi-Fidelity Pareto Optimization (Successive Halving).

# Survival Analysis: Winning Configs

- **pair format dominates:** 0–10 score per model.
- **CoT hurts:** explanation/answer lowers accuracy, raises cost.
- **Temperature  $\approx 0$  (0.0, 0.01) wins.**
- **Order-swap averaging boosts stability.**



**Figure 4:** Hyperparameter Survival Analysis (Top 100 Configurations).

# Results: LMSys Data

- 3 judges (Small <10B, Medium <32B, Large <72B) on 3,000 test instructions.
- Fine-tuned PandaLM-7B, JudgeLM-7B fail (distribution shift).
- **Ours-large**: Human Agreement 0.49  $\approx$  GPT-4o-mini (0.50).
- **Cost**: \$0.48 vs. \$1.2 per 1k.

| Judge       | Human agr. ( $\uparrow$ ) | Cost per 1K ann. ( $\downarrow$ ) |
|-------------|---------------------------|-----------------------------------|
| Random      | 0.33 +/- 0.01             | -                                 |
| Length      | 0.42 +/- 0.01             | -                                 |
| PandaLM-7B  | 0.38 +/- 0.01             | 6.0                               |
| JudgeLM-7B  | 0.42 +/- 0.01             | 8.6                               |
| Arena-Hard  | 0.50 +/- 0.01             | 1.2                               |
| Ours-small  | 0.45 +/- 0.01             | 0.21                              |
| Ours-medium | 0.47 +/- 0.01             | 0.48                              |
| Ours-large  | 0.49 +/- 0.01             | 0.48                              |

**Figure 5:** Comparison of judges on LMSys test instructions

# Results: PandaLM & Arena-Hard

- **PandaLM: Ours-small** (<10B) 0.67 > dedicated 70B PandaLM.
- **Arena-Hard: Ours-medium** Spearman **0.93** @ **\$0.48**/1k vs. GPT-4 0.90 @ **\$50**/1k.

| Judge       | Human agr. (↑) |
|-------------|----------------|
| Random      | 0.33           |
| Length      | 0.60           |
| GPT-3.5     | 0.63           |
| GPT-4       | 0.67           |
| PandaLM-7B  | 0.62           |
| PandaLM-70B | 0.67           |
| Ours-small  | 0.67           |
| Ours-medium | 0.78           |
| Ours-large  | 0.76           |

**Table 1:** Comparison with PandaLM on PandaLM test set

| Judge               | Sp. corr. (↑) | Cost per 1K ann. (↓) |
|---------------------|---------------|----------------------|
| Length              | 0.50 +/- 0.21 | -                    |
| Arena-hard + Claude | 0.82 +/- 0.12 | 75.0                 |
| Arena-hard + GPT4   | 0.90 +/- 0.06 | 50.0                 |
| Ours-small          | 0.81 +/- 0.10 | 0.21                 |
| Ours-medium         | 0.93 +/- 0.05 | 0.48                 |
| Ours-large          | 0.86 +/- 0.09 | 0.48                 |

**Table 2:** Spearman correlation between win-rates

- **Limited generalizability:** only 3 Decoder-only families; MoE unproven.
- **CoT rejection uncontrolled:** may be “Lost in the Middle” in sub-10B models, not judging per se.
- **Early-filtering risk:** Successive Halving assumes question homogeneity — hard-task specialists may be wrongly early-stopped.
- **Goodhart’s Law:** optimizing for human agreement amplifies hidden human biases.
- **Cost oversimplified:** ignores hardware setup, VRAM, orchestration for 32B/72B.



**Paper 6:**  
**Trust or Escalate: LLM Judges with Provable  
Guarantees**

- **Paper:** *TRUST OR ESCALATE: LLM Judges with Provable Guarantees for Human Agreement*
- **Goal:** Control LLM judging reliability with *mathematically provable* guarantees — not just heuristics.
- **Category:** Scalability Limits & Benchmark Robustness.

# Motivation: Why Heuristics Fail

- **Large-scale evaluation** (e.g., 1M samples): human labeling is infeasible.
- **GPT-4 as judge**: expensive heuristic — suffers from systematic bias (length, position) and over-confidence.
- **Key question**: how to make LLM judging *reliable* with a *provable risk bound*?

# Mathematical Framework: Trust or Escalate

- **Core idea:** the model should *not* always vote — if uncertain, it should abstain ( $\rightarrow$  escalate).
- **Conditional output:**

$$(f_{LM}, c_{LM})(x) = \begin{cases} f_{LM}(x) & \text{if } c_{LM}(x) \geq \lambda \\ \emptyset & \text{otherwise} \end{cases}$$

- $f_{LM}$ : preference vote.  $c_{LM}$ : confidence (0–1).  $\lambda$ : threshold.
- **Risk guarantee:**

$$P(f_{LM}(x) = y_{human} \mid c_{LM}(x) \geq \lambda) \geq 1 - \alpha$$

- User sets error risk  $\alpha$  and failure probability  $\delta$ .

# Calibration: Fixed-Sequence Testing

- **Calibration set**  $D_{cal}$ : small human-labeled set (e.g., 500 samples) to find optimal  $\hat{\lambda}$ .
- **Binomial upper bound**: model errors on  $D_{cal}$  follow a binomial distribution — compute strictest upper bound  $\hat{R}^+(\lambda)$  at confidence  $1 - \delta$ .
- **Fixed-Sequence Testing**:
  - Start from highest threshold ( $\lambda = 0.999$ ) and move down.
  - Stop at last point where  $\hat{R}^+(\lambda) \leq \alpha$ .
- Much less conservative than Bonferroni correction; maintains higher coverage.

# Innovation 1: Simulated Annotators

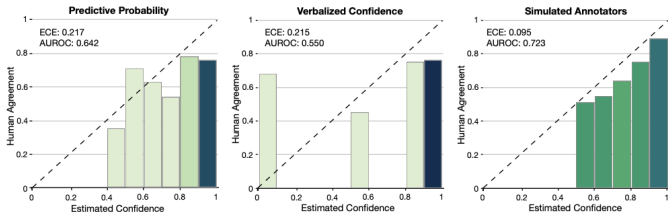
- **Problem:** standard confidence metrics (predictive probability, verbalized confidence) are over-confident (high ECE).
- **Solution:** simulate diverse human preferences via few-shot prompting.
- **Mechanism:**
  - Query model  $N$  times with different in-context examples, each mimicking a specific human  $j$ .
  - Ensemble internal vote:

$$c_{LM}(x) = \max_y \frac{1}{N} \sum_{j=1}^N p_{LM}(y \mid x; \text{examples}_j)$$

- Disagreement among simulators  $\rightarrow$  low confidence  $\rightarrow$  escalate. ECE reduced by  $\sim 50\%$ .

# Innovation 1 : Illustration

| Dataset              |                             | AlpacaEval   |              |              |              | TL;DR        |              |              |              |
|----------------------|-----------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| Method               |                             | Acc.         | ECE ↓        | AUROC        | AUPRC        | Acc.         | ECE ↓        | AUROC        | AUPRC        |
| <i>GPT-4-turbo</i>   | Predictive Probability      | 0.724        | 0.217        | 0.642        | 0.852        | 0.760        | 0.196        | 0.731        | 0.890        |
|                      | Verbalized Confidence       | 0.724        | 0.215        | 0.550        | 0.774        | 0.760        | 0.194        | 0.548        | 0.792        |
|                      | Randomized Annotators       | 0.720        | 0.113        | 0.705        | 0.866        | 0.779        | 0.079        | 0.734        | 0.905        |
|                      | Simulated Annotators (Maj.) | 0.730        | 0.106        | 0.718        | 0.873        | 0.783        | 0.062        | <b>0.755</b> | <b>0.921</b> |
|                      | Simulated Annotators (Ind.) | <b>0.734</b> | <b>0.095</b> | <b>0.723</b> | <b>0.877</b> | <b>0.788</b> | <b>0.039</b> | <b>0.755</b> | <b>0.921</b> |
| <i>GPT-3.5-turbo</i> | Predictive Probability      | 0.644        | 0.293        | 0.581        | 0.691        | 0.667        | 0.228        | 0.653        | 0.786        |
|                      | Verbalized Confidence       | 0.644        | 0.306        | 0.505        | 0.595        | 0.667        | 0.211        | 0.607        | 0.716        |
|                      | Simulated Annotators (Ind.) | <b>0.694</b> | <b>0.058</b> | <b>0.632</b> | <b>0.793</b> | <b>0.725</b> | <b>0.043</b> | <b>0.704</b> | <b>0.842</b> |
| <i>Mistral-7B-it</i> | Predictive Probability      | 0.618        | 0.374        | 0.457        | 0.579        | 0.661        | 0.306        | 0.613        | 0.735        |
|                      | Verbalized Confidence       | 0.618        | 0.414        | 0.490        | 0.627        | 0.661        | 0.335        | 0.578        | 0.680        |
|                      | Simulated Annotators (Ind.) | <b>0.684</b> | <b>0.075</b> | <b>0.632</b> | <b>0.772</b> | <b>0.696</b> | <b>0.103</b> | <b>0.654</b> | <b>0.807</b> |



**Figure 6:** Reliability plot for confidence estimation methods

## Innovation 2: Cascaded Architecture

- **Goal:** cut massive cost of multi-testing with large models (GPT-4).
- **Cascade pipeline:**

Mistral-7B-it  $\longrightarrow$  GPT-3.5-turbo  $\longrightarrow$  GPT-4-turbo

- Each layer: if  $c(x) \geq \lambda_i \rightarrow$  vote and stop; else  $\rightarrow$  escalate.
- Abstention policy at each layer auto-tuned based on global risk budget — no manual chain tuning.



# Experimental Setup

- **Models:** Mistral-7B-it  $\rightarrow$  GPT-3.5-turbo  $\rightarrow$  GPT-4-turbo.
- **Stats:**  $|D_{cal}| = 500$ ,  $\delta = 0.1$  (90% success guarantee), averaged over 1000 runs.
- **Datasets:**
  - **TL;DR:** summarization task (multiple human labels  $\rightarrow$  use independent simulators).
  - **Chatbot Arena:** real-world chat comparisons (single human label  $\rightarrow$  majority-based simulators).

- **Risk control:** GPT-4 alone achieves 0% success at 90% agreement target; proposed method passes *all* risk levels.
- **Coverage at 90% agreement:** 55.7% of data covered; half handled by Mistral + GPT-3.5.
- **At 70% agreement:** coverage reaches 96%; Mistral alone handles 84%.

# Results: TL;DR

| Method                               | Evaluator Composition (%) |               |             | Coverage (%) | Guarantee Success Rate (%) |
|--------------------------------------|---------------------------|---------------|-------------|--------------|----------------------------|
|                                      | Mistral-7B                | GPT-3.5-turbo | GPT-4-turbo |              |                            |
| No Selection                         | 0.0                       | 0.0           | 100.0       | 100.0        | 0.0                        |
| Heuristic Selection                  | 0.0                       | 0.0           | 100.0       | 89.6         | 42.0                       |
| Cascaded Heuristic Selection         | 59.6                      | 15.0          | 25.5        | 64.6         | 0.0                        |
| Point-Estimate Calibration           | 100.0                     | 0.0           | 0.0         | 5.6          | 54.7                       |
|                                      | 0.0                       | 100.0         | 0.0         | 9.4          | 79.0                       |
|                                      | 0.0                       | 0.0           | 100.0       | 57.7         | 47.5                       |
| <b>Cascaded Selective Evaluation</b> | <b>28.3</b>               | <b>28.2</b>   | <b>43.5</b> | <b>55.7</b>  | <b>90.8</b>                |

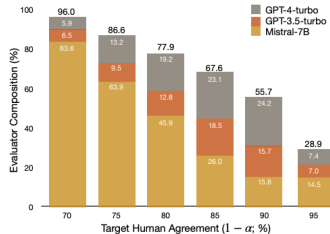
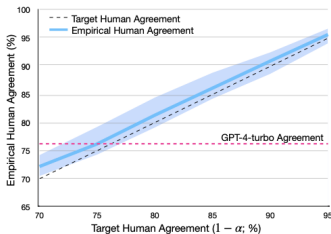


Figure 7: TL;DR results

# Results: Chatbot Arena

- **Guaranteed 85% agreement:** system success rate = 91.0% (matches  $\delta = 0.1$  expectation); all baselines near 0%.
- **Total coverage:** 63.2%.
- **Load distribution:** Mistral-7B (23.7%), GPT-3.5 (58.8%), GPT-4 (only 17.5%).
- **Key:** >82% of evaluations completed without GPT-4, yet outperform blind GPT-4.

# Results: Chatbot Arena

| Method                               | Evaluator Composition (%) |                      |                    | Coverage (%) | Guarantee Success Rate (%) |
|--------------------------------------|---------------------------|----------------------|--------------------|--------------|----------------------------|
|                                      | <i>Mistral-7B</i>         | <i>GPT-3.5-turbo</i> | <i>GPT-4-turbo</i> |              |                            |
| No Selection                         | 0.0                       | 0.0                  | 100.0              | 100.0        | 0.0                        |
| Heuristic Selection                  | 0.0                       | 0.0                  | 100.0              | 95.2         | 0.1                        |
| Cascaded Heuristic Selection         | 57.1                      | 15.2                 | 27.7               | 79.7         | 0.3                        |
| Point-Estimate Calibration           | 100.0                     | 0.0                  | 0.0                | 0.0          | 0.0                        |
|                                      | 0.0                       | 100.0                | 0.0                | 40.5         | 57.2                       |
|                                      | 0.0                       | 0.0                  | 100.0              | 60.9         | 54.4                       |
| <b>Cascaded Selective Evaluation</b> | <b>23.7</b>               | <b>58.8</b>          | <b>17.5</b>        | <b>63.2</b>  | <b>91.0</b>                |

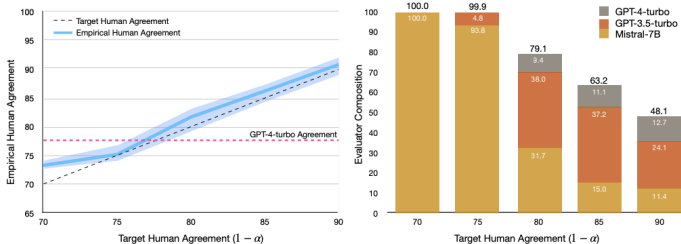


Figure 8: ChatArena results

- **$D_{cal}$  sensitivity:** 500 samples  $\rightarrow$  conservative upper bound  $\rightarrow$  reduced real coverage; hidden human-labeling cost.
- **Token overhead:** Simulated Annotators with  $N$  judges  $\times K$  few-shot examples  $\rightarrow$  huge context load even for cheap models.
- **Binary pairwise only:** the binomial bound does not extend to scalar scoring (e.g., 1–10 ratings).
- **Mild distribution-shift assumption:** stability under extreme shift (summarization  $\rightarrow$  coding/math) unproven; i.i.d. violation collapses the guarantee.
- **Escalation cost ambiguity:** unclear how  $\emptyset$  samples are handled; human re-labeling cost can erode financial gains.

## **Category 4: Structured Evaluation Protocols**

---

**Paper 1:**  
**RocketEval: Efficient Automated LLM**  
**Evaluation via Grading Checklist**



- **Problem:** GPT-4-class judges are useful but expensive, API-dependent, private, and hard to reproduce.
- **Observation:** Small open models fail mostly because they miss detailed analysis, not because they cannot answer simple checks.
- **Core Idea:** Convert one broad evaluation into many narrow checklist questions.
- **Goal:** Make a lightweight local judge reliable enough for scalable open-ended evaluation.

# Why Small Judges Fail

- Direct CoT judging still leaves weak models uncertain and inconsistent.
- Two dominant error sources are **high uncertainty** and **position bias**.
- RocketEval reduces the task from “give a global score” to “verify checklist items.”
- This makes judging more interpretable: every final score has item-level evidence.

# RocketEval Framework

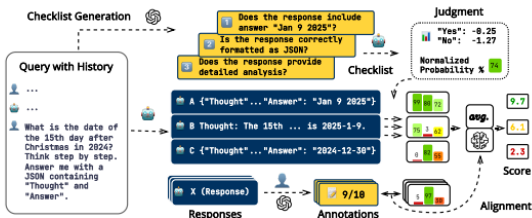


Figure 2: Illustration of **RocketEval** framework for automated LLM evaluation. The framework consists of three components: Checklist Creation, Checklist Grading and Score Prediction.

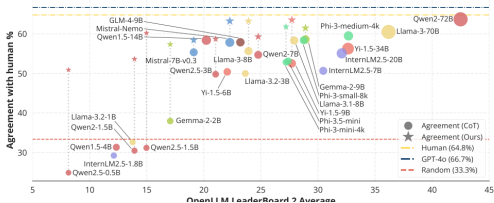
- Three stages: checklist creation, checklist grading, score prediction.
- The pipeline separates *criterion design* from *criterion verification*.

## Method: Checklist QA Instead of Global Judgment

- **Checklist creation:** generate focused criteria for correctness, completeness, relevance, and instruction-following.
- **Checklist grading:** ask the lightweight judge whether each criterion is satisfied.
- **Score prediction:** average item scores, or learn weights from supervised labels.

$$\hat{p} = \frac{p_{\theta}(\text{Yes} \mid x, y, c)}{p_{\theta}(\text{Yes} \mid x, y, c) + p_{\theta}(\text{No} \mid x, y, c)}$$

# Key Result: Small Judges Become Competitive



- RocketEval improves many small judges over ordinary CoT judging.
- With Gemma-2-2B, it reports **0.965** correlation with human preferences and over **50×** cost reduction.

## Critical Review & Bridge to Paper 2

- **Strength:** transparent item-level evidence; cheaper and more reproducible than closed judge APIs.
- **Limitation:** checklist quality still depends on a strong generator; weak or biased checklists can mislead the judge.
- **Limitation:** supervised reweighting still needs reliable labels.
- **Bridge:** Paper 1 improves *structured criteria*; Paper 2 improves *comparative reasoning coverage*.

**Paper 2:**  
**Crowd Comparative Reasoning**  
**Unlocking Comprehensive Evaluations for**  
**LLM-as-a-Judge**

- **Problem:** LLM judges often produce incomplete CoT judgments when comparing only two candidate answers.
- **Why it matters:** shallow reasoning misses subtle differences and can produce unreliable preferences.
- **Existing fixes:** majority voting and longer rubrics help, but they are either costly or response-unaware.
- **Main Question:** Can extra crowd responses expose hidden details in the two candidate responses?



## Core Idea: Crowd-Based Comparative Evaluation

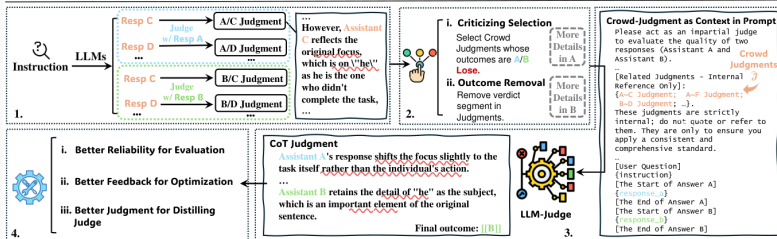
- Generate several **crowd responses** for the same instruction.
- Compare each candidate response against these crowd responses.
- Use the resulting **crowd judgments** as context for the final A/B judgment.
- The crowd acts as evaluation anchors: different responses reveal different strengths and weaknesses.

# CCE Framework

**Instruction:** Rewrite the following sentence in a more concise. Although he had been studying for several hours he had not finished the task.

**Resp A:** Despite studying for hours, the task remained incomplete.

**Resp B:** Despite studying for several hours, he had not finished the task.



- Crowd judgments are selected, cleaned, and inserted into the final judge prompt.
- The final judge produces a richer CoT and a more reliable preference label.

# Main Results Across Judge Benchmarks

| Model                         | REWARD<br>BENCH | HELPSTEER2  | MTBENCH<br>HUMAN | JUDGE<br>BENCH | EvalBIAS    | Avg.        |
|-------------------------------|-----------------|-------------|------------------|----------------|-------------|-------------|
| <b>GPT-4o</b>                 |                 |             |                  |                |             |             |
| <i>Vanilla</i>                | 85.2            | 66.1        | 82.1             | 66.3           | 68.5        | 73.6        |
| <i>LongPrompt</i>             | 86.9            | 67.3        | 81.8             | 63.5           | 70.5        | 74.0        |
| <i>EvalPlan</i>               | 88.7            | 65.5        | 81.4             | 62.9           | 74.4        | 74.6        |
| <i>16-Criteria</i>            | 87.3            | 69.1        | 82.8             | 66.6           | 73.7        | 75.9        |
| <i>Maj@16</i>                 | 87.9            | 68.9        | 82.4             | 68.6           | 75.5        | 76.7        |
| <i>Agg@16</i>                 | 88.1            | 68.7        | 82.6             | 67.2           | 77.9        | 76.9        |
| <i>CCE-random@16</i>          | 91.2            | 69.5        | 83.1             | 68.9           | 80.1        | 78.6        |
| <i>CCE@16</i>                 | <b>91.8</b>     | <b>70.6</b> | <b>83.6</b>      | <b>70.4</b>    | <b>85.0</b> | <b>80.3</b> |
| <b>Qwen 2.5 7B-Instruct</b>   |                 |             |                  |                |             |             |
| <i>Vanilla</i>                | 78.2            | 60.7        | 76.1             | 58.3           | 57.4        | 66.1        |
| <i>CCE@16</i>                 | <b>80.4</b>     | <b>64.2</b> | <b>76.7</b>      | <b>64.0</b>    | <b>79.4</b> | <b>72.9</b> |
| <b>Qwen 2.5 32B-Instruct</b>  |                 |             |                  |                |             |             |
| <i>Vanilla</i>                | 87.4            | <b>72.3</b> | 79.0             | 68.9           | 71.1        | 75.7        |
| <i>CCE@16</i>                 | <b>90.8</b>     | 72.1        | <b>82.1</b>      | <b>70.6</b>    | <b>80.5</b> | <b>79.2</b> |
| <b>Qwen 2.5 72B-Instruct</b>  |                 |             |                  |                |             |             |
| <i>Vanilla</i>                | 85.2            | <b>69.5</b> | 79.5             | 68.3           | 68.5        | 74.0        |
| <i>CCE@16</i>                 | <b>93.7</b>     | 68.5        | <b>88.9</b>      | <b>75.7</b>    | <b>85.9</b> | <b>82.7</b> |
| <b>Llama 3.3 70B-Instruct</b> |                 |             |                  |                |             |             |
| <i>Vanilla</i>                | 86.4            | 70.4        | 81.1             | 67.1           | 70.6        | 75.1        |
| <i>CCE@16</i>                 | <b>91.7</b>     | <b>71.3</b> | <b>83.5</b>      | <b>69.7</b>    | <b>79.2</b> | <b>79.1</b> |

- CCE improves average accuracy across five benchmarks.
- The paper reports an average **6.7%** gain over strong judging baselines.

# Beyond Evaluation: Distillation and SFT

- **Judge distillation:** longer, higher-quality CCE rationales train smaller judge models more effectively.
- **Crowd rejection sampling:** CCE helps select better training samples from candidate response pools.
- **Scaling behavior:** accuracy improves as more crowd comparisons are used at inference time.
- **Practical value:** CCE is not only an evaluator; it also improves training data selection.

# Critical Review & Relation to Paper 1

- **Strength:** directly targets incomplete CoT reasoning by forcing comparison against diverse anchors.
- **Strength:** useful for evaluation, judge distillation, and rejection sampling.
- **Limitation:** extra crowd responses increase inference cost and prompt length.
- **Limitation:** if crowd responses are low-quality or homogeneous, they may add noise instead of insight.
- **Takeaway:** RocketEval decomposes criteria; CCE expands comparison context. Together, they make LLM judges more structured and more informative.